SPECIFICATION REPORT

Digital Signal Encoder, Scrambler & Decoder Implementation

ITT 036 | Autumn 2026 | Programming Assignment 1 | October 21, 2026

GitHub: https://github.com/star100rabh/Data_Com_Assignment.git

Saurabh Patel 2024BITE045	**Hemendra Kumar** 2024BITE011	**Mohit Moriya** 2024BITE024

1. Language and Libraries

Python 3.8+: complies with non-MATLAB course guidelines.

NumPy: vectorized time-domain signal arrays, quantization zones and step arrays.

Matplotlib: high-resolution waveforms, boundary markers and plots.

Tkinter: interactive desktop GUI (extra credit).

2. System Architecture and Modules

data_generator.py: stream generation; 4/8-zero injection (HDB3/B8ZS).

palindrome.py: Manacher's O(N) palindrome detection.

line_coding.py: NRZ-L, NRZ-I, Manchester, Differential Manchester, AMI.

scrambling.py: B8ZS (000VB0VB), HDB3 and O(N) zero-run finder.

modulation.py: PCM quantization/encoding; Delta Modulation staircase.

decoder.py: physical waveform parameter and direct image-pixel decoders.

plotter.py / gui.py / main.py: visualizer, desktop GUI and unified CLI.

3. Key Engineering Assumptions

Voltage levels: normalized bipolar signaling: High = +1.0 V, Low = -1.0 V, Zero = 0.0 V.

Sampling: bit duration T_b = 1.0 s; 100 sample points/bit to model continuous physical transitions without step discontinuities, as described in the original specification.

Initial states: NRZ-I and Differential Manchester use a pre-transmission reference of +1.0 V. AMI begins with negative pulse history, so the first data '1' produces a positive (+1.0 V) pulse.

Waveform and image decoding (extra credit): receivers reconstruct bitstreams without accessing source bit memory. Physical decoding samples voltage levels and boundary derivatives from (t, v) arrays. **decode_from_image** segments waveform-trace pixels in saved PNG plots, decoding from encoded visual signal parameters.

Descrambling: B8ZS/HDB3 decoders detect bipolar violations (V, same polarity as the previous pulse) and balancing pulses (B), restoring substituted patterns to the original consecutive-zero sequences.

4. Competitive Coding Complexity Analysis

Longest palindromic sequence: Manacher's algorithm uses O(N) time and O(N) space. '#' delimiters unify odd/even palindromes; center-mirror symmetry (2C - i) reuses previous radii, avoiding nested if/else ladders and brute-force $O(N^3)$/$O(N^2)$ checks.

Longest zero-run sequence: a single-pass window tracks current and maximum zero runs in O(N) time and O(1) auxiliary space.

5. Execution Guide and Automated Verification

Command	Function and scope	Reported result
python3 main.py	Interactive CLI: digital/analog input, line coding, scrambling, decoding.	Verified (CLI flow)
python3 gui.py	Desktop visualizer: Tkinter + Matplotlib interactive canvas.	Verified (GUI window)
python3 test_suite.py	Automated checks across all seven assignment components.	PASS: 7/7 (100%)
python3 decoder.py	Physical parameter and direct PNG image-pixel decoding checks.	MATCH: 100%
python3 palindrome.py	Manacher's edge-case checks: odd/even, random and zero-run streams.	PASS: O(N)

Results above are retained from the supplied specification; tests were not rerun during report formatting.

6. References

1. B. A. Forouzan, *Data Communications and Networking*, 5th ed., McGraw-Hill, 2012; Ch. 4: Digital Transmission; Ch. 5: Analog Transmission.

2. IEEE Standard 802.3-2018, *IEEE Standard for Ethernet: Physical Layer and Management Parameters*.

3. G. Manacher, "A New Linear-Time 'On-Line' Algorithm for Finding the Smallest Initial Palindrome of a String," *Journal of the ACM*, vol. 22, no. 3, pp. 346-351, 1975.